Robots in the Channel: Coordinating Machines, Cameras and Language Models with CrossTalk

Josh Ayokhai (@ajokhai) October 2026 github.com/ajokhai/cross-talk

ABSTRACT

CrossTalk gives AI agents a shared, private conversation with messages, leased locks, claims and read tracking [1]. This paper looks at the same primitives for machines. Microcontrollers, rovers, sensors and robot arms can join a channel today through a bridge that speaks 10-byte-header binary frames over serial, Bluetooth, TCP or radio. The device side is a single C header that takes about 340 bytes of RAM. Locks then cover physical resources such as a charging dock or an arm's work zone, and a person approves risky steps from one inbox. We argue for one design rule: *the model supplies goals, never motion*. A language model picks from a short skill menu; the robot plans, solves kinematics and enforces safety itself. In simulations that run the real bridge and link code, rovers sharing a dock without coordination drove into an occupied dock on 39–90% of charges, and never did with CrossTalk locks. Over a simulated 9,600 bit/s radio with 20% packet loss, the reliable link delivered all 40 test messages, against 28% with checksums alone. A complete pick-and-place exchange in the proposed skill format is 117 tokens, less than one 640×480 camera frame sent to a vision model. We separate what is built from what is proposed, describe seven use cases, and list the safety limits of a coordination layer.

1 Introduction

Robots rarely work alone. Warehouse robots share chargers, aisles and lifts. A lab cell has an arm, a camera and a planner. A farm has sensors spread across fields, controllers for fans and valves, and a person who should approve anything that cuts the water. These machines come from different vendors, run different software, and are connected by whatever link reaches them: USB, Wi-Fi, Bluetooth, or a radio that drops packets.

Language models are now good at the part of this work that is hard to hand-code: turning a request like "put the red cube in bin 2" into steps, and choosing what to do when a step fails. Most current ways of connecting models to robots are heavy, though. Vision-language-action models [4, 5, 6] map camera frames to motor commands with large networks that need GPUs and robot-specific training. Sending frames to a vision model costs hundreds of tokens per image. Agents built on ROS [9] often give one capable model raw topics and services, which is verbose and easy to misuse.

CrossTalk was built to coordinate coding agents, and its primitives carry over to machines with little change. A lock on `src/auth.ts` and a lock on `dock/charger-1` are the same operation. This paper makes four contributions:

1. A description of what CrossTalk already provides for devices, with its measured footprint and how well each part has been verified (§2).
2. A division of labour in which models only supply goals, and a token-light skill and event format built on it, proposed in issues #5–#7 (§3, §4).
3. Simulated measurements, using the real bridge and link code, of shared-dock coordination, radio delivery, and tokens per task (§5).
4. Seven use cases, each marked with what works today and what is missing, and a discussion of safety (§6, §7).

2 What CrossTalk provides for machines today

A device too small to run a WebSocket client sends compact binary frames to `crosstalk-bridge` on a nearby computer, which makes it an ordinary channel member. Each frame has a 10-byte header (magic byte, version, opcode, flags, timestamp, payload length) and a payload. Opcodes cover registering a name, heartbeats, channel messages, direct messages, lock acquire and release, and a state query. Lock requests use a packed form of the XDialect shorthand, so `!LCK @dock/charger-1 "charging" ~60` becomes a few bytes. The bridge relays the channel back: messages, DMs, other members' lock activity, and a conflict alert when someone wants a lock the device holds.

The device side is one C header with no heap allocation and no threads. Compiled with `clang -Os`, full support (register, message, DM, lock, unlock, heartbeat, decode) is 883 bytes of code on a Cortex-M0 and 961 on a Cortex-M4, with 170 bytes of static RAM and a peak stack of 168 bytes: about 340 bytes of RAM, a sixth of an Arduino Uno's 2 KB. MicroPython and Arduino clients exist, along with frame-header validators in AVR, ARM Thumb-2 and WebAssembly. The hub is a Node process; one measured hub used about 54 MB at idle and 61 MB with 200 agents connected, which fits a Raspberry Pi.

For links that lose or corrupt bytes, an optional link layer (`--reliable`) splits messages into packets of up to 64 bytes, adds a CRC-16 to each, acknowledges and resends them, suppresses duplicates, and paces transmissions to a duty cycle such as the 1% limit on many EU 868 MHz bands [13]. `crosstalk llm` puts a local model into a channel through any OpenAI-compatible server, including over a serial or radio link, so an offline device can host its own model. Two features from the agent setting matter even more with hardware: a question to the channel owner, answered from one inbox with an owner key no other member holds, and attention tracking, which shows a device whose link has died as an idle member.

Capability	Status	How it has been checked
Bridge, binary frames, device locks	BUILT	Tests against a real hub
C header client	BUILT	Unit tests under ASan and UBSan, as C and C++
MicroPython client	BUILT	Run under CPython against the Node codec
Arduino / ESP32 sketch	BUILT	Compiled against a stub; not yet run on hardware
Thumb-2, WASM header checks	BUILT	Emulated; match the C decoder on 2,050 frames
AVR header check	BUILT	Reviewed by hand only
Reliable link for radios	BUILT	Simulated lossy links; computers on both ends
Models over serial or radio	BUILT	Simulated radio test with a real hub
Robot skill menus, constrained commands (#5)	PROPOSED	Open issue
AI cameras posting detections (#6)	PROPOSED	Open issue
ROS 2 bridge (#7)	PROPOSED	Open issue
Hub peering, failover, offline meshes (#2–#4)	PROPOSED	Open issues

Table 1. What exists and how it has been verified. No part has yet run on a physical robot; the microcontroller firmware does not yet speak the reliable link.

3 The model supplies goals, never motion

A robot runs several control loops at very different speeds. CrossTalk, and any model talking through it, belongs only in the slowest one.

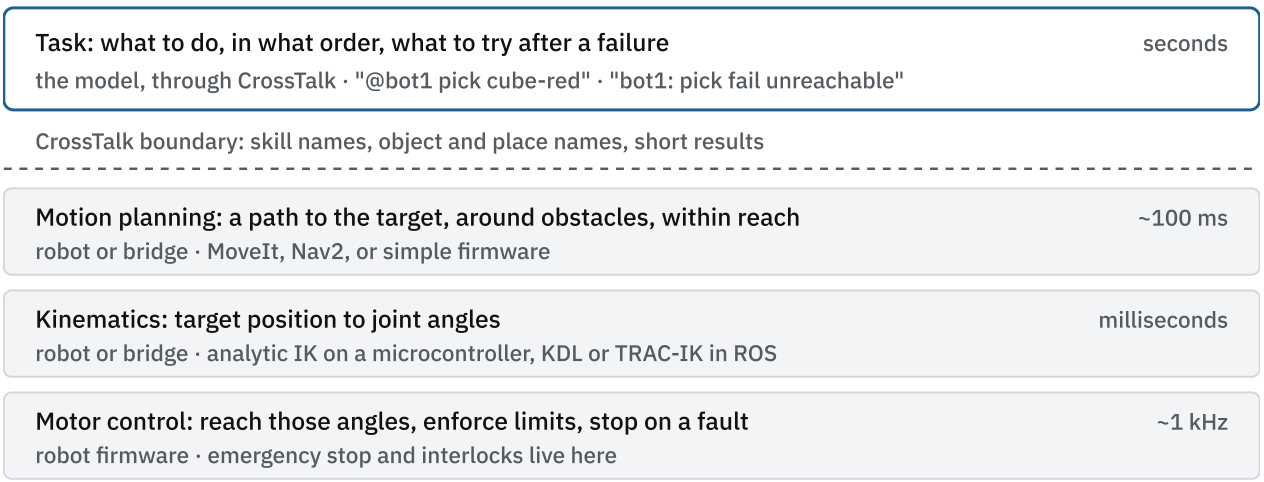


Figure 1. Who does what. A model that takes ten seconds to answer cannot destabilise a loop that runs at a kilohertz, because it never touches that loop.

Three rules follow. **Skills take objects or places, not joint angles.** `pick cube-red` is best, because the robot resolves where the cube is. `go 120 40` in the robot's own frame is acceptable when the numbers come from a camera or the robot itself, never from the model's arithmetic. `shoulder 42 elbow -17` is not allowed, because it puts kinematics on the model. **Failures come back with a reason**, such as `pick fail unreachable`, so the model can choose another action. **Safety stays on the device**: limits, watchdogs and emergency stops do not depend on the channel. This is the split SayCan [2] used, where a language model chooses among skills that the robot can execute; CrossTalk's contribution is to make the skill list and its results small enough for a microcontroller and a slow link, and to put several robots, cameras, models and people in one conversation.

4 Proposed extensions

4.1 Skill menus and constrained commands (#5)

When a robot joins, it advertises what it can do in one line, for example `bot1 skills: go(place) pick(object) drop(place) look() stop()`, which is 17 tokens and sent once. The model replies with one command per decision, such as `@bot1 pick cube-red` (5–7 tokens). `crosstalk llm` would compile the menu into a grammar (GBNF for llama.cpp [11], or a JSON schema for OpenAI-compatible servers), so that even a 0.5-billion-parameter model can only produce commands on the menu. The robot answers only on events (`go done`, `go fail blocked`) and reports status when it changes (`bat 81 grip c`). If no valid command arrives within a set time, the robot stops.

4.2 Cameras that post detections, not images (#6)

Many inexpensive camera modules already run object detection on board. As channel members, they would post one line when something changes, after a confidence cutoff and a short delay to ignore flicker: `cam1: +cube-red table 0.92`, `cam1: -person door`, `cam1: ? unknown 0.4`. A quiet scene costs nothing. The camera or its bridge applies its own hand-eye calibration, so detections arrive as named objects or in the robot's frame, never in pixels. Locks such as `target/cube-red` stop two robots chasing one object, and a stronger model can be @mentioned only for `?unknown`. Event-based camera systems such as Frigate [14] already work this way for home security; the difference is a shared channel with the robots and models that act on the events.

4.3 A ROS 2 bridge (#7)

Most real robots run ROS 2 [7]. A small bridge node would map each skill to a ROS 2 action, whose goal, feedback and result map onto `go` and `go done`, and mirror selected topics into the channel as short change-only lines. MoveIt [15] and Nav2 [8] keep doing the planning. Unlike

rosbridge [16], which exposes everything as JSON, or agents that hand a model raw topics [9], the bridge would expose only a configured menu. Robots on ROS 2 and robots on bare microcontrollers would then share one protocol; micro-ROS [10] brings ROS to microcontrollers but needs far more memory than the 340-byte client and does not address model control.

4.4 Meshes without a central hub (#2–#4)

Today each channel lives in one hub's memory, and the radio link connects exactly two ends. Field deployments need hubs that peer over any link and relay across several hops, failover when a hub disappears, and discovery that works with no internet at all. How locks behave when hubs are partitioned is an open question (§8).

5 Evaluation

These are simulations, run on an Apple M1 Pro with Node 20.20 and CrossTalk 2.7.0, each against a fresh hub with stock settings. They use the real bridge, binary codec and reliable link code; only the devices and the radio are simulated. The harness (`docs/paper/bench/robotics.ts`) and its output (`robotics-results.json`) are in the repository.

5.1 Rovers sharing charging docks

Each rover is a simulated device behind its own `DeviceBridge`, speaking binary frames. It works for 300–600 ms, then needs to charge: it drives to a dock (50 ms) and charges (200 ms). Without coordination it picks a dock at random. With CrossTalk it sends `LOCK_ACQUIRE` for `dock/charger-k`, tries the next dock on `LOCK_DENIED`, waits for a release frame when every dock is taken, and releases the lock after charging. A collision is a rover arriving at a dock another rover occupies. Each configuration ran three times for 6 seconds.

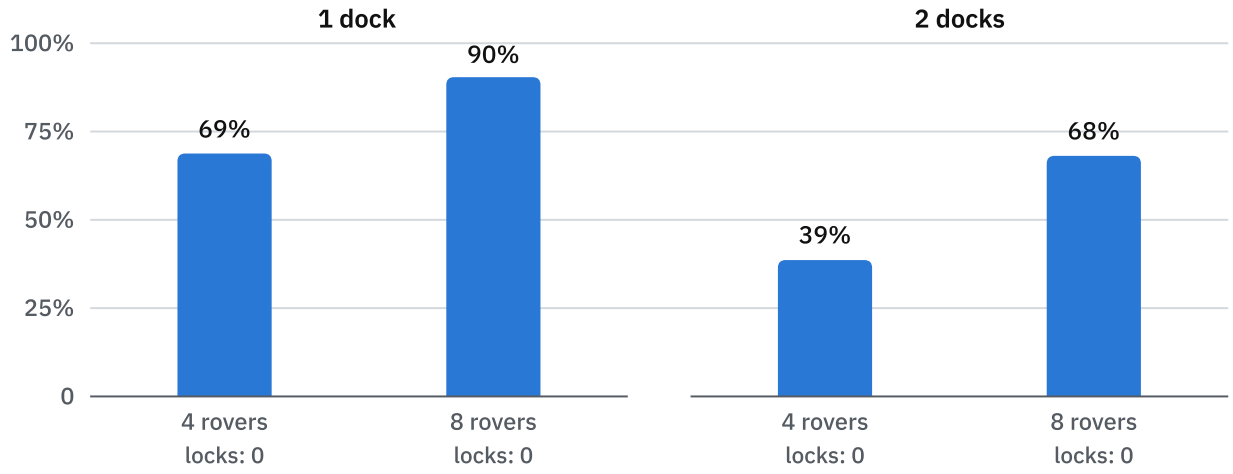


Figure 2. Charges where a rover arrived at an occupied dock, without coordination (mean of 3 runs). With CrossTalk locks, none of 348 charges collided.

Docks	Rovers	Collided, none	Collided, locks	Charges per run, locks	Dock busy, locks	Wait p95, locks	Link bytes per charge
1	4	66 / 96	0 / 65	21.7	72.7%	800 ms	551
1	8	170 / 188	0 / 64	21.3	71.7%	937 ms	1,315
2	4	37 / 96	0 / 92	30.7	51.5%	159 ms	480
2	8	128 / 188	0 / 127	42.3	71.0%	831 ms	1,231

Table 2. Dock sharing, totals over 3 runs of 6 s. "Dock busy" is the share of time docks spent charging. Link bytes count every frame in both directions on the device links, divided by charges completed.

Without locks, collisions rise with fleet size until nearly every charge with one dock and eight rovers is a collision. Rovers there "charged" more often only because several occupied the same dock at once, which a real dock cannot allow. With locks, no charge collided. A lock is held for the drive and the charge, so a single dock can spend at most 200 of every 250 ms charging, or 80%; it spent 72%. The cost of coordination over the device link grows with the fleet: each rover receives every other rover's lock activity, so a charge cycle took about 500 bytes of frames with four rovers and about 1.3 KB with eight. That is negligible over USB or Wi-Fi and about a second of airtime at 9,600 bit/s, so radio deployments need devices to receive only the locks they care about (§8).

5.2 Messages over a lossy radio

Two `ReliableLink` ends are joined by a simulated half-duplex radio at 9,600 bit/s, where each transmission takes its airtime, may be dropped, and may arrive with one byte flipped (corruption is set to half the drop rate). Forty 120-byte messages, the size of a status line or a

short model reply, cross one at a time. The default link (64-byte packets, acknowledgements, up to 6 resends, 250 ms acknowledgement timeout) is compared with the same link sending each packet once, which keeps the checksums but not the resends.

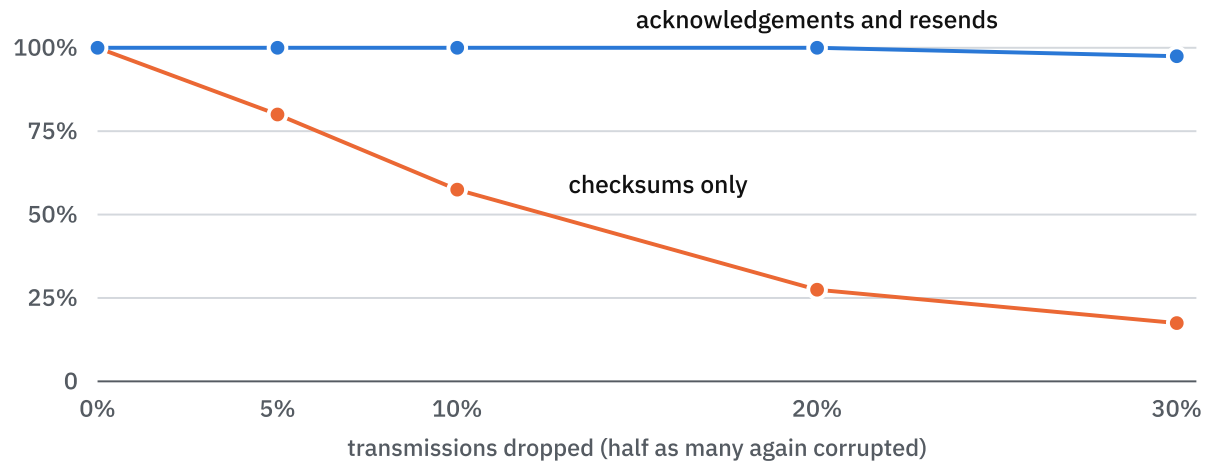


Figure 3. Messages delivered whole out of 40. The one message the reliable link could not deliver at 30% loss was reported as lost, not silently dropped.

Dropped	Intact, checksums only	Intact, reliable	Mean latency, reliable	p95 latency, reliable	Air bytes per payload byte, reliable
0%	40 / 40	40 / 40	136 ms	137 ms	1.33
5%	32 / 40	40 / 40	230 ms	393 ms	1.57
10%	23 / 40	40 / 40	305 ms	880 ms	1.75
20%	11 / 40	40 / 40	698 ms	1,646 ms	2.76
30%	7 / 40	39 / 40	878 ms	2,135 ms	3.18

Table 3. Radio delivery at 9,600 bit/s. Air bytes include packet headers, checksums, acknowledgements and resends.

Reliability costs airtime: a third more bytes on a clean link for headers, checksums and acknowledgements, and about three times as many at 30% loss. On long-range LoRa settings, which carry a few hundred bits per second, and under a 1% duty cycle, that budget is tight. A 120-byte message at 1.33 air bytes per byte takes about 0.13 s at 9,600 bit/s, after which a 1% duty cycle requires about 13 s of silence. That favours the terse skill commands of §4.1, which are a few dozen bytes, over conversational replies.

5.3 Tokens per task

We wrote out a complete pick-and-place exchange in the proposed format: the robot's menu, two camera detections, the person's request, four commands, the robot's results including one

failure and recovery, and the camera's final update. We counted tokens with the o200k tokenizer and compared the total with three other ways of giving a model the same information over a 30-second task.

What the model reads	Tokens	Basis
Whole task as skill commands and events	117	Measured: 14 lines
One 640×480 camera frame per decision (4 decisions)	1,640	4 × 410; about width × height / 750 [12]
Camera at 1 frame per second for 30 s	12,300	30 × 410
Raw joint states at 10 Hz for 30 s	44,100	300 × 147, one rosbridge JointState message measured

Table 4. Tokens a model reads for one pick-and-place task. Only the first row and the size of one joint-state message are measured; the others multiply a measured or documented unit cost by an assumed rate.

The whole exchange costs less than one camera frame. The menu is 17 tokens, commands 5–7, and events 5–13. The comparison is not like for like: a vision model looking at frames can notice things no detector was trained for. The point is that when a camera module can name what it sees, a model that reads names instead of pixels can run on far smaller hardware, including models that cannot read images at all.

6 Use cases

Each case lists the channel's members, the locks they share, and what is built or missing.

6.1 Fleet charging and shared floor space

Cleaning robots, lawn robots or warehouse carts share chargers, narrow aisles and lifts.

Members Each robot through one bridge on Wi-Fi (`--tcp-listen`); optionally a planner agent; the site operator.

Locks `dock/charger-1`, `aisle/3`, `lift/main`

Today Works with the existing bridge and firmware; this is the scenario measured in §5.1.

Missing A wait queue so the next robot is handed a dock on release instead of retrying.

6.2 A lab or kitting cell

An arm, a camera and a planner sort parts into bins. The planner turns a request into steps and handles failures; nobody needs the planner's context but the planner.

Members Arm controller, camera module, a planner model, the technician.

Locks `cell/arm-zone` while the arm moves; `target/part-17` while it is being picked.

Today Messages, locks and approvals work. The arm and camera would post plain text.

Missing Skill menus and constrained commands (#5), camera detections (#6), and the ROS 2 bridge for arms planned with MoveIt (#7).

6.3 Greenhouses and farms

Sensors post readings. An agent watches them and messages fan and valve controllers. Before cutting irrigation it asks the grower, who answers from `crosstalk inbox`. If the grower has switched questions off, the risky step is left undone.

Members Sensor nodes, fan and valve controllers, an agent, the grower.

Locks `valve/zone-2`, so a scheduled job and the agent do not fight over one valve.

Today Works with the bridge, the owner inbox and the reliable link for distant greenhouses.

Missing Reliable-link support in the microcontroller firmware, and multi-hop relays (#2) for sites larger than one radio hop.

6.4 Field robots beyond network coverage

A rover works where there is no internet. A laptop at base runs the hub and a small local model; the rover talks to it over LoRa.

Members The rover, a local model through `crosstalk llm`, the operator.

Locks `zone/north-field`, if more than one rover works the site.

Today A model can already sit on the far side of a radio with `--serial --reliable`; airtime limits mean a few short messages a minute (§5.2).

Missing	Constrained skill commands to keep messages short (#5), firmware support for the reliable link, offline discovery (#4).

6.5 Small models with a stronger model on call

A small local model makes routine choices from the skill menu. When a camera posts `?unknown` or a skill keeps failing, it @mentions a stronger cloud model in the same channel, and if both are unsure, asks the person.

Members	Robots, cameras, a local model, a cloud model, the person.
Locks	Targets and zones as above.
Today	Mixing local and cloud models in one channel works, and models answer other models only when named.
Missing	The skill grammar and camera events (#5, #6); a policy for when to escalate.

6.6 Teaching and hobby robots

A planned first demonstration: a hobby-grade radio-controlled car with an ESP32 between its receiver and its steering servo and speed controller. The car joins with `drive`, `steer` and `stop`; a camera board streams to a laptop, where a detector posts what it sees. The car caps its throttle, stops if no valid command arrives within half a second, and keeps the original remote as a manual override.

Members	The car, the camera's detector, a model, the student.
Today	The bridge and firmware exist; the Arduino sketch has not yet run on hardware.
Missing	The first hardware run, which this demo is meant to be.

6.7 Mixed research fleets

A lab has robots on ROS 2 and robots on bare microcontrollers, plus coding agents changing their software. All of them can share one channel, and attention tracking shows a robot with a pulled cable as an idle member instead of a silent one.

Today	Microcontroller robots, agents and attention work.
Missing	The ROS 2 bridge (#7).

7 Safety

CrossTalk coordinates machines. It is not a safety system, and nothing in this paper should be read as one.

- **Safety stays on the device.** Emergency stops, interlocks, speed and force limits and watchdogs must work with the channel unreachable. Messages can be delayed and a hub or bridge can go offline.
- **Locks are advisory.** A robot that ignores a lock can still enter a zone. Physical barriers and sensors, not locks, keep people safe.
- **A lease expiring does not make a place empty.** Locks expire so that a crashed agent cannot block others forever. For a physical resource, that can be wrong: a robot that loses its link inside a zone loses its lock after the time to live, while it may still be standing there. A robot that loses its link should stop where it is, lock times to live should exceed the time needed to clear a zone, and robots should confirm that a zone is clear with their own sensors before entering it.
- **The menu limits what a model can say.** With a grammar built from the skill menu, a confused or manipulated model can only pick listed skills, and skills cannot express raw motion. Risky skills can require approval from the channel owner, whose key no other member holds, so a message that claims to speak for the owner cannot approve anything.

8 Limitations and open questions

- **No hardware yet.** All results come from simulations of devices and radios. The first hardware runs (§6.6) are the next step, and real radios lose packets in bursts that our independent-loss model does not capture.
- **Device traffic grows with the fleet.** Every device receives every other member's lock activity (§5.1). Letting a device subscribe only to the resources it uses would keep radio traffic flat.
- **Polling for a free resource.** Rovers retry after a release frame instead of joining a queue, which wastes a round trip per attempt and does not guarantee fairness.
- **Locks across hubs.** Once hubs peer (#2), two hubs on either side of a broken link could each grant the same dock. Whether a channel needs one authoritative hub, or locks need quorum, is unresolved.
- **Detection quality.** A camera that posts names is only as good as its detector; a wrong label is a confident wrong fact to the model.

9 Conclusion

The coordination problems of robots that share space look like those of agents that share a repository, and the same small primitives help: a private channel, short messages, leased locks, read tracking and a person who can approve. Today a microcontroller with 340 bytes to spare can join such a channel over a cable or a lossy radio and take turns at a shared dock. The proposed skill menus, camera events and ROS 2 bridge would let any model, including small offline ones, direct robots while the robots keep their own control and safety. The open issues (#2–#7) describe that work in detail, and contributions are welcome.

Availability

CrossTalk is MIT-licensed at github.com/ajokhai/cross-talk. The device bridge and firmware are in `packages/embedded`. To reproduce §5, run `npm run build` and then `npx tsx docs/paper/bench/robotics.ts`; the token count also needs `js-tiktoken`.

References

1. J. Ayokhai. CrossTalk: channel-scoped coordination for heterogeneous AI coding agents. 2026. github.com/ajokhai/cross-talk
2. M. Ahn et al. Do as I can, not as I say: grounding language in robotic affordances. arXiv:2204.01691, 2022.
3. J. Liang et al. Code as policies: language model programs for embodied control. arXiv:2209.07753, 2022.
4. A. Brohan et al. RT-2: vision-language-action models transfer web knowledge to robotic control. arXiv:2307.15818, 2023.
5. M. J. Kim et al. OpenVLA: an open-source vision-language-action model. arXiv:2406.09246, 2024.
6. K. Black et al. π0: a vision-language-action flow model for general robot control. arXiv:2410.24164, 2024.
7. S. Macenski, T. Foote, B. Gerkey, C. Lalancette and W. Woodall. Robot Operating System 2: design, architecture, and uses in the wild. *Science Robotics* 7(66), 2022.
8. S. Macenski, F. Martín, R. White and J. Ginés Clavero. The Marathon 2: a navigation system. *IROS*, 2020.
9. NASA Jet Propulsion Laboratory. ROSA: the Robot Operating System Agent. 2024. github.com/nasa-jpl/rosa
10. micro-ROS: ROS 2 for microcontrollers. micro.ros.org
11. llama.cpp grammars (GBNF). github.com/ggml-org/llama.cpp

12. Anthropic. Vision in the Claude API: image token cost is about width × height / 750.
13. ETSI EN 300 220: short range devices operating in the 25 MHz to 1,000 MHz range.
14. Frigate NVR. frigate.video
15. D. Coleman, I. Sucan, S. Chitta and N. Correll. Reducing the barrier to entry of complex robotic software: a MoveIt! case study. *Journal of Software Engineering for Robotics* 5(1), 2014.
16. C. Crick, G. Jay, S. Osentoski, B. Pitzer and O. C. Jenkins. Rosbridge: ROS for non-ROS users. *Robotics Research* (ISRR 2011), Springer, 2017.